Evaluation Contexts

The upper rules are computation rules, the lower congruence rules.

Computation rules:

$$\begin{array}{rll} v := & n & Values \\ v_1 \oplus v_2 \rightarrow & & (v_1 \oplus v_2) \\ \texttt{ifz}(0; t_1, t_2) \rightarrow & & t_1 \\ \texttt{ifz}(v; t_2, t_2) \rightarrow & & t_2 \text{ if } n \neq 0 \end{array}$$

Congruence rules:

$$\frac{t_1 \rightarrow t_1'}{t_1 \oplus t_2 \rightarrow t_1' \oplus t_2} \quad \frac{t_2 \rightarrow t_2'}{v \oplus t_2 \rightarrow v \oplus t_2'} \quad \frac{t_1 \rightarrow t_1'}{\texttt{ifz}(t_1; t_2, t_3) \rightarrow \texttt{ifz}(t_1'; t_2, t_3)}$$

Congruence rules re-formulated using *evaluation contexts*:

$$\begin{array}{ll} E ::= & \\ & [\,] \\ & E \oplus t \\ & v \oplus E \\ & \texttt{ifz}(E; t_2, t_3) \end{array}$$

Evaluation

Derivation Trees

$$\frac{\dfrac{3 * 4 \rightarrow 12}{(3 * 4) - 5 \rightarrow 12 - 5}}{\texttt{ifz}((3 * 4) - 5;\ (7 * 7) + 15,\ 4 + 4)\ 7 \rightarrow \texttt{ifz}(12 - 5;\ (7 * 7) + 15,\ 4 + 4)}$$

Normal Forms

A term which is in normal form and is a value is quite simple: $t_1 = 42$.

However, a term which is in normal form and not a value does not exist. The grammer of the language only allows for three terms: values, concatenation between terms using binary operations, or `ifz` which consumes 3, possibly different terms.

To not be in normal form, a term has to be found which cannot be evaluated. A value would fit this requirement, but the question also asks for a term that is not a value. Therefore, a concatenation between binary operations or usage of `ifz` has to be found.

Let's first look at the binary operations. Either t_1 is not a value, congruence rule 1 applies.

Induction on Derivation Trees

(i) $t = v$

$$\texttt{size}(t) = \texttt{size}(v) = 1 = \texttt{size(t')} \tag{1}$$

(ii) $t = t_1 \oplus t_2, t_1 \to^* v_1$

$$\begin{aligned}
\texttt{size}(t_1 \oplus t_2) &= \texttt{size}(t_1) + \texttt{size}(t_2) + 1 && (2)\\
&\geq \texttt{size}(v_1) + \texttt{size}(t_2) + 1 && (3)\\
&= \texttt{size}(t') && (4)\\
\Leftrightarrow \texttt{size}(t_1) &\geq \texttt{size}(v_1) && (5)\\
\texttt{size}(t_1) &\geq 1 && (6)
\end{aligned}$$

Analog for $t = t_1 \oplus t_2, t_2 \to^* v_2$

(iii) $t = \texttt{ifz}(t_1; t_2, t_3), t_1 \to^* v_1$

$$\begin{aligned}
\texttt{size}(\texttt{ifz}(t_1; t_2, t_3)) &= \texttt{size}(t_1) + \texttt{size}(t_2) + \texttt{size}(t_3) + 1 && (7)\\
&\geq \texttt{size}(v_1) + \texttt{size}(t_2) + \texttt{size}(t_3) + 1 && (8)\\
&= \texttt{size}(t') && (9)\\
\Leftrightarrow \texttt{size}(t_1) &\geq \texttt{size}(v_1) && (10)\\
\texttt{size}(t_1) &\geq 1 && (11)
\end{aligned}$$

Types and Programming Languages

Is OCaml mandatory or can we use it at home to veryfy the contents of the book? Also, are the questions mandatory? Chapter 4 was rather short compared to the beginnings of the book till Chapter 3, and I feel like I have to force myself to come up with a question?